

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns
```

```
In [2]: df=pd.read_csv("HousingData.csv")
df.head()
```

```
Out[2]:
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	

```
In [3]: df.dtypes
```

```
Out[3]: CRIM      float64
ZN          float64
INDUS       float64
CHAS        float64
NOX         float64
RM          float64
AGE         float64
DIS         float64
RAD         int64
TAX         int64
PTRATIO     float64
B           float64
LSTAT       float64
MEDV        float64
dtype: object
```

```
In [4]: df.count()
```

```
Out[4]: CRIM      486
        ZN        486
        INDUS    486
        CHAS     486
        NOX      506
        RM       506
        AGE      486
        DIS      506
        RAD      506
        TAX      506
        PTRATIO  506
        B        506
        LSTAT    486
        MEDV     506
        dtype: int64
```

```
In [5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   CRIM    486 non-null    float64
 1   ZN      486 non-null    float64
 2   INDUS   486 non-null    float64
 3   CHAS    486 non-null    float64
 4   NOX     506 non-null    float64
 5   RM      506 non-null    float64
 6   AGE     486 non-null    float64
 7   DIS     506 non-null    float64
 8   RAD     506 non-null    int64   
 9   TAX     506 non-null    int64   
10  PTRATIO  506 non-null    float64
11  B        506 non-null    float64
12  LSTAT    486 non-null    float64
13  MEDV     506 non-null    float64
dtypes: float64(12), int64(2)
memory usage: 55.5 KB
```

```
In [6]: (df.isnull()).sum()
```

```
Out[6]: CRIM      20
        ZN        20
        INDUS     20
        CHAS      20
        NOX        0
        RM         0
        AGE       20
        DIS        0
        RAD        0
        TAX        0
        PTRATIO    0
        B          0
        LSTAT     20
        MEDV       0
        dtype: int64
```

```
In [7]: (df==0).sum()
```

```
Out[7]: CRIM      0
        ZN      360
        INDUS     0
        CHAS    452
        NOX       0
        RM        0
        AGE       0
        DIS       0
        RAD       0
        TAX       0
        PTRATIO   0
        B         0
        LSTAT     0
        MEDV      0
        dtype: int64
```

```
In [8]: print(df["ZN"],df["CHAS"])
```

```

0      18.0
1       0.0
2       0.0
3       0.0
4       0.0
...
501     0.0
502     0.0
503     0.0
504     0.0
505     0.0
Name: ZN, Length: 506, dtype: float64 0      0.0
1       0.0
2       0.0
3       0.0
4       0.0
...
501     0.0
502     0.0
503     0.0
504     0.0
505     0.0
Name: CHAS, Length: 506, dtype: float64

```

```
In [9]: df = df.dropna()
df.isnull().sum()
```

```
Out[9]: CRIM      0
ZN          0
INDUS      0
CHAS       0
NOX        0
RM         0
AGE        0
DIS        0
RAD        0
TAX        0
PTRATIO    0
B          0
LSTAT      0
MEDV       0
dtype: int64
```

```
In [10]: df.RAD.unique()
```

```
Out[10]: array([ 1,  2,  3,  5,  4,  8,  6,  7, 24])
```

```
In [11]: df.RAD.value_counts()
```

```
Out[11]: RAD
         24    100
         4     91
         5     90
         3     31
         2     21
         8     19
         6     18
         1     12
         7     12
         Name: count, dtype: int64
```

```
In [13]: bins=(0,4,25)
group_names=["bad_railway_acess" , "good_railway_acess"]
df["Railway_Index"]=pd.cut(df.RAD , bins=bins,labels=group_names)
df["Railway_Index"]
```

```
Out[13]: 0      bad_railway_acess
         1      bad_railway_acess
         2      bad_railway_acess
         3      bad_railway_acess
         5      bad_railway_acess
         ...
        499    good_railway_acess
        500    good_railway_acess
        502    bad_railway_acess
        503    bad_railway_acess
        504    bad_railway_acess
         Name: Railway_Index, Length: 394, dtype: category
         Categories (2, object): ['bad_railway_acess' < 'good_railway_acess']
```

```
In [14]: from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
In [15]: label_quality=LabelEncoder()
df["Railway_Index"]=label_quality.fit_transform(df["Railway_Index"])
df.head(10)
```

Out[15]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63
5	0.02985	0.0	2.18	0.0	0.458	6.430	58.7	6.0622	3	222	18.7	394.12
7	0.14455	12.5	7.87	0.0	0.524	6.172	96.1	5.9505	5	311	15.2	396.90
8	0.21124	12.5	7.87	0.0	0.524	5.631	100.0	6.0821	5	311	15.2	386.63
10	0.22489	12.5	7.87	0.0	0.524	6.377	94.3	6.3467	5	311	15.2	392.52
11	0.11747	12.5	7.87	0.0	0.524	6.009	82.9	6.2267	5	311	15.2	396.90
12	0.09378	12.5	7.87	0.0	0.524	5.889	39.0	5.4509	5	311	15.2	390.50

In [16]:

```
df.astype('int32').dtypes
df.head()
```

Out[16]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63
5	0.02985	0.0	2.18	0.0	0.458	6.430	58.7	6.0622	3	222	18.7	394.12

In [17]:

```
X=df.drop("Railway_Index",axis=1)
y=df['Railway_Index']
```

In [18]:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
```

In [19]:

```
X_train, X_test, y_train , y_test= train_test_split(X,y,test_size=0.25,random_state
```

In [20]:

```
X_train[:1]
```

Out[20]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B
20	1.25179	0.0	8.14	0.0	0.538	5.57	98.1	3.7979	4	307	21.0	376.57

```
In [21]: rfc=RandomForestClassifier(n_estimators=100)
rfc.fit(X_train, y_train)
pred_rfc=rfc.predict(X_test)
pred_rfc[:2]
```

Out[21]: array([0, 0])

```
In [22]: from sklearn.metrics import accuracy_score
cm=accuracy_score(y_test,pred_rfc)
cm
```

Out[22]: 1.0

In []: